

Laboratorio - Construir un script de Python para administrar equipos de Webex.

Objetivos

- Parte 1: Ejecutar la máquina virtual de DEVASC.
- Parte 2: Obtenga su token de acceso de Webex.
- Parte 3: Probar su token de acceso.
- Parte 4: Gestionar personas en equipos de Webex.
- Parte 5: Administrar salas en equipos de Webex.
- Parte 6: Gestionar membresías en equipos de Webex.
- Parte 7: Administrar mensajes en equipos de Webex.

Aspectos básicos/Situación

En este laboratorio, usará las API de Webex para autenticar, administrar personas, administrar salas, administrar membresías a salas y enviar un mensaje.

Recursos necesarios

- Una computadora con el sistema operativo de su elección con la aplicación de Webex instalada.
- Virtual Box o VMWare.
- Máquina virtual de DEVASC.

Instrucciones

Parte 1: Ejecutar la Máquina virtual DEVASC.

Si aún no ha completado el **Laboratorio: Instalar el Entorno de Laboratorio de Máquina Virtual**, hágalo ahora. Si se ha completado ya, inicie la máquina virtual DEVASC.

Nota: Parte del **Laboratorio - Instalar el entorno de laboratorio de máquinas virtuales** es instalar la aplicación Webex y agregar al menos una persona a su lista de contactos. Esto debe completarse antes de continuar con este laboratorio.

Parte 2: Obtener su token de acceso de Webex.

En esta parte, se registra o inicia sesión en su cuenta de Webex, explora la documentación de la API, obtiene su token de acceso y, a continuación, prueba el token de acceso que utilizará en sus llamadas a la API.

Paso 1: Iniciar sesión en Webex.

- a. Abra el navegador web Chromium.
- b. Vaya al sitio web del desarrollador de Webex: <https://developer.webex.com/>
- c. Inicie sesión si ya tiene una cuenta. Si no es así, siga el proceso de registro.

Paso 2: Explorar la documentación de la API.

- Haga clic en **Documentación**.
- Webex tiene muchas API que puede usar en sus aplicaciones. Haga clic en la **Referencia de API** para expandir su submenú.
- Explore todas las variedades de llamadas API. En este laboratorio, usará la documentación de API para **Personas, Salas, Membresía y Mensaje**.

Paso 3: Obtener su token de acceso.

- Aún en **Documentación**, desplácese hasta el inicio, si lo haya necesario, de clic en **Comenzar** ubicado bajo la **sección API REST**.
- En **Cuentas y autenticación**, observe que Webex admite un token de acceso personal. Se requiere un token de autenticación para todas las llamadas a la API REST. Haga clic en el ícono Copiar en Token de Acceso Personal.

Nota: Un token de acceso personal proporciona acceso a su cuenta a cualquier persona que la conozca. Asegúrese de mantenerlo en secreto.

Nota: Recibirá un mensaje indicando que el token es válido durante una cantidad de tiempo establecida, que era de 12 horas en el momento en que se escribió este laboratorio. Deberá obtener un nuevo token y actualizar sus scripts de Python si regresa a este laboratorio después de que caduque su token.

- Copie su token de acceso personal en un archivo de texto para utilizarlo más adelante en este laboratorio.

Parte 3: Prueba el token de acceso.

Puede probar su token de acceso dentro de la documentación de OpenAPI en el sitio del desarrollador. Sin embargo, usará su token en scripts de Python. Por lo tanto, debe probar que funciona en un script, también.

Paso 1: Prueba su token de acceso en el sitio del desarrollador.

Puede probar su token de acceso dentro de la documentación de OpenAPI en <https://developer.webex.com>.

- Vuelva a su navegador y haga clic en **Documentación**, si es necesario.
- En **Referencia de API**, haga clic en **Personas** y, a continuación, haga clic en **Obtener mis propios detalles**.
- En el panel **Probar** en el extremo derecho, observe que su token ya está al tope.
- Puede hacer clic en **Probar** o **Ejecutar** para verificar su acceso. Verá la **respuesta** con su información personal.
- Haga clic en **Solicitar** para ver la URL completa utilizada para enviar la solicitud GET. Usará esta URL en el siguiente paso de su script de Python.
- En la sección central, puede revisar toda la documentación de las **Propiedades de la respuesta**.

Paso 2: Utilizar una secuencia de comandos de Python para probar su token de acceso.

- Abra Código VS. A continuación, haga clic en **Archivo > Abrir carpeta...** y navegue al directorio **devnet-src/webex-teams**. Haga clic en **Aceptar**.
- En el panel **EXPLORADOR**, ahora debería ver todos los archivos.py marcadores de posición que utilizará en este laboratorio. Haga clic en el archivo **authentciation.py**.
- Coloque el código siguiente en el archivo. Asegúrese de reemplazar **su_token_aquí** con su token de acceso personal que copió en el paso anterior.

Solicitudes de importación.

Importar JSON.

```
access_token = 'su_token_here'
url = 'https://webexapis.com/v1/people/me'
headers = (
    'Authorization': 'Bearer {}'.format(access_token)
)
res = requests.get(url, headers=headers)
print(json.dumps(res.json(), indent=4))
```

- d. Guarde y ejecute el archivo. Debería obtener el mismo resultado que vio en la documentación de OpenAPI.

Nota: Los valores de algunas de las claves se han truncado en la salida siguiente.

```
devasc @labvm: ~/labs/devnet-src/webex-teams$ python3 authentication.py
{
  "id": "Y21z... UYM2u",
  "emails": [
    "your-email@example.com"
  ],
  "PhoneNumbers": [],
  "displayName": "Your-First-Name Your-Last-Name",
  "nickName": "Your-Nick-Name",
  "firstName": " Your-First-Name",
  "lastName": "Your-Last-Name",
  "avatar": "https://9643-417f-9974...6baa4~1600 ",
  "OrgID": "Y21zy2... UxMgy",
  "created": "2012-06-15T20:23:12.529Z",
  "lastActivity": "2020-06-02T20:16:52.111Z",
  "status": "active",
  "type": "person"
}
devasc @labvm: ~/labs/devnet-src/webex-teams$
```

Parte 4: Gestionar personas en equipos de Webex.

En Webex, las personas son usuarios registrados. A través de la API, puede recuperar una lista de personas, puede crear una persona, recuperar los detalles de una persona individual, actualizar una persona y eliminar una persona.

Paso 1: Buscar la documentación de la API para ver los detalles de un usuario registrado de Webex.

- Vuelva al sitio web developer.webex.com. En la parte **Referencia de API > Personas**, haga clic en el **Método de lista de personas**.
- En **Parámetros de consulta**, busque el parámetro de **correo electrónico**. Este es el parámetro que utilizará para encontrar un usuario específico en su organización. Alternativamente, puede usar el parámetro **DisplayName**, si conoce el nombre exacto. Puede utilizar la función Try it.

Paso 2: Utilizar un script de Python para enumerar los detalles de un usuario registrado de Webex.

- En Código VS, haga clic en el archivo **list-people.py**.
- Coloque el código siguiente en el archivo. Asegúrese de reemplazar **su_token_aquí** con su token de acceso personal y el **usuario @example.com** por un usuario real registrado en Webex de su organización

Solicitudes de importación.

Importar JSON.

```
access_token = 'su_token_here'
url = 'https://webexapis.com/v1/people'
headers = (
    'Authorization': 'Bearer {}'.format(access_token),
    'Content-Type': 'application/json'
)
params = {
    'email': 'user@example.com'
}
res = requests.get(url, headers=headers, params=params)
print(json.dumps(res.json(), indent=4))
```

- Guarde y ejecute el script. Debe de obtener un resultado similar al siguiente. Si recibe un mensaje como, **{'message': 'Dirección de correo electrónico no válida. '...**, significa que no reemplazó el parámetro de correo electrónico vacío por una dirección de correo electrónico legítima para un usuario registrado de Webex. El valor de la clave **id** se utilizará en la próxima llamada a la API.

Nota: Los valores de algunas de las claves se han truncado en la salida siguiente.

```
devasc @labvm: ~/labs/devnet-src/webex-teams$ python3 list-people.py
```

```
{
  "NotFoundIds": null,
  "items": [
    {
      "id": "y2l... 2I", # Usará este valor en el siguiente paso
      "emails": [
        "user@example.com"
      ],
      "PhoneNumbers": [
        {
          "type": "mobile",
          "value": "+1234567690"
        }
      ],
      "DisplayName": "DisplayName",
      "nickName": "nickName",
      "FirstName": "FirstName",
      "lastName": "lastName",
      "avatar": "https://9643-417f-9974...6baa4~1600 ",
      "OrgID": "Y2lzy... uxMgy",
      "created": "2012-06-15T20:39:19.726Z",
      "LastActivity": "2020-06-04T 13:57:01 .688Z",
```

```
        "status": "active",
        "type": "person"
    }
]
}
devasc @labvm: ~/labs/devnet-src/webex-teams$
```

Paso 3: Enumerar detalles administrativos adicionales para una persona.

- a. Si es administrador de Webex, puede obtener detalles adicionales de una persona utilizando el valor de la clave **id** de persona en su llamada a la API. Agregue el siguiente código a su script **list-people.py**. Reemplace **previous_id_here** con el valor para **id** de la llamada API anterior.

```
person_id = 'previous_id_here'
url = 'https://webexapis.com/v1/people/{}'.format(person_id)
headers = (
    'Authorization': 'Bearer {}'.format(access_token),
    'Content-Type': 'application/json'
)
res = requests.get(url, headers=headers)
print(json.dumps(res.json(), indent=4))
```

- b. Guarde el archivo y ejecútelo como no administrador, obtendrá información que es muy similar al paso anterior.

```
devasc @labvm: ~/labs/devnet-src/webex-teams$ python3 list-people.py
{
  <first API call output omitted>
}
{
  "id": "Y2l... 2I",
  "emails": [
    "user@example.com"
  ],
  "PhoneNumbers": [
    {
      "type": "mobile",
      "value": "+1234567890"
    }
  ],
  "DisplayName": "DisplayName",
  "nickName": "nickName",
  "FirstName": "FirstName",
  "lastName": "lastName",
  "avatar": "https://9643-417f-9974...6baa4~1600 ",
  "OrgID": "Y2l... MGY",
  "created": "2012-06-15T20:39:19.726Z",
  "LastActivity": "2020-06-04T 14:39:36 .535Z",
  "status": "active",
  "type": "person"
}
devasc @labvm: ~/labs/devnet-src/webex-teams$
```

Parte 5: Administrar salas en Webex.

Las salas, también llamadas espacios en la interfaz de usuario, permiten a las personas enviar mensajes y archivos para colaborar virtualmente en lugares de reunión colectivos. En esta parte, mostrará una lista de salas, creará una sala y obtendrá los detalles de una sala.

Paso 1: Localizar e investigar la documentación de API para salas.

- Vuelva al sitio web developer.webex.com. En **Referencia de API**, haga clic en **salas**.
- Explore las distintas llamadas API que puede realizar con la API de **Salas**.
- Haga clic en la solicitud GET para las **Listas de Salas** y explore los **parámetros de consulta**.

Paso 2: Utilizar una secuencia de comandos de Python para enumerar todas las salas de un usuario autenticado.

- Para este paso, deberá ser miembro de al menos una sala. Una conversación con otra persona se considera una sala en Webex.
- En Código VS, haga clic en el archivo **list-rooms.py**.
- Coloque el código siguiente en el archivo. Asegúrese de reemplazar **su_token_aquí** con su token de acceso personal.

Solicitudes de importación.

```
access_token = 'your_token_here'
url = 'https://webexapis.com/v1/rooms'
headers = (
    'Authorization': 'Bearer {}'.format(access_token),
    'Content-Type': 'application/json'
)
params= {'max': '100'}
res = requests.get(url, headers=headers, params=params)
print (res.json ())
```

- Guarde y ejecute el archivo. Su resultado va a ser diferente del siguiente. Solo hay una sala en la lista. Los valores de ID se han truncado.

```
devasc @labvm: ~/labs/devnet-src/webex-teams$ python3 list-rooms.py
{'items': [{'id': 'Y2l...ZTE0', 'title': 'User Name', 'type': 'direct', 'isLocked':
False, 'lastActivity': '2020-06-01T16:34:56.536Z', 'creatorId': 'Y2lz...yM2U',
'created': '2020-06-01T16:30:21.816Z', 'ownerId': 'Y2lz...xMGY'}]
# salas adicionales mostradas hasta el valor "máximo".
[]
devasc @labvm: ~/labs/devnet-src/webex-teams$
```

Paso 3: Localizar e investigar la documentación de API para su publicación en la API de salas.

- Volver al sitio web developer.webex.com. En **Referencia de API**, haga clic en **salas**, si es necesario.
- La API de salas tiene un método POST para **Crear una sala**. Haga clic en el vínculo de **Parámetros de consulta** para consultar cuáles están disponibles. En el script, usará el parámetro de **título** requerido.

Paso 4: Utilizar una secuencia de comandos de Python para crear una sala.

- En Código VS, haga clic en el archivo **create-rooms.py**.

- b. Coloque el código siguiente en el archivo. Asegúrese de reemplazar **su_token_aquí** con su token de acceso personal. Observe que se trata de una solicitud POST y utiliza el parámetro **title**.

Solicitudes de importación.

```
access_token = 'your_token_here'
url = 'https://webexapis.com/v1/rooms'
headers = (
    'Authorization': 'Bearer {}'.format(access_token),
    'Content-Type': 'application/json'
)
params= {'title': 'DevNet Associate Training! '}
res = requests.post(url, headers=headers, json=params)
print (res.json ())
```

- c. Guarde y ejecute el archivo. Debería obtener una respuesta similar a la siguiente. Los valores de ID se han truncado. El **ID** de la sala y el **título** están resaltados. Copie el ID de la sala y guárdelo en un archivo de texto para utilizarlo en el resto de este laboratorio.

```
devasc @labvm: ~/labs/devnet-src/webex-teams$ python3 create-rooms.py
{'id': 'Y2l... gNm', 'title': 'DevNet Associate Training! ', 'type': 'group',
 'isLocked': False, 'lastActivity': '2020-06-04T 16:50:19 .371Z', 'CreatorID': 'Y2l...
M2U', 'created': '2020-06-04T 16:50:19 .371Z', 'ownerId': 'Y2l... MGY'}
```

- d. ¡En su aplicación Webex, verifique que ahora vea la sala de **formación de asociados de DevNet!** Actualmente eres el único miembro.

Paso 5: Utilizar un script de Python para obtener detalles de la sala.

¿Qué pasa si quieres empezar una reunión en tu nueva sala? Puede realizar otra llamada GET para recuperar la dirección del Protocolo de inicio de sesión (SIP), la dirección URL de la reunión y los números de teléfono de acceso telefónico.

- a. En Código VS, haga clic en el archivo **get-room-details.py**.
- b. Coloque el código siguiente en el archivo. Reemplace **su_token_aquí** con su token de acceso personal. Reemplace **su_id_de sala** con el valor que obtuvo en el paso anterior.

Solicitudes de importación.

```
access_token = 'your_token_here'
room_id = 'your_room_id'
url = 'https://webexapis.com/v1/rooms/{}/meetingInfo'.format(room_id)
headers = (
    'Authorization': 'Bearer {}'.format(access_token),
    'Content-Type': 'application/json'
)
res = requests.get(url, headers=headers)
print (res.json ())
```

- c. Guarde y ejecute el archivo. Debería obtener una respuesta similar a la siguiente. Los valores se han truncado.

```
devasc @labvm: ~/labs/devnet-src/webex-teams$ python3 get-room-details.py
{'RoomId': 'Y2l... gnm', 'MeetingLink': 'https://cisco.webex.com/j/3272...a837',
 'sipAddress': '162...468@cisco.webex.com', 'MeetingNumber': '162... 0468',
 'CallIntollFreeNumber': '+1-866-... -9903', 'CallInTollNumber': '+1-408-... -6800'}
```

```
devasc @labvm: ~/labs/devnet-src/webex-teams$
```

Parte 6: Gestionar membresías en Webex.

En esta parte, usará la API de membresía para agregar a alguien a su sala.

Paso 1: Localizar e investigar la documentación de API para las suscripciones.

- Vuelva al sitio web developer.webex.com. En **Referencia de API**, haga clic en **Membresías**.
- Explore las distintas llamadas API que puede realizar con la API de **Membresías**.
- Haga clic en la solicitud GET para **la lista de membresía** y explore los **parámetros de consulta**.

Paso 2: Utilizar una secuencia de comandos de Python para enumerar los miembros de la sala.

- En Código VS, haga clic en el archivo **list-memberships.py**.
- Coloque el código siguiente en el archivo. Reemplace **su_token_aquí** con su token de acceso personal. Reemplace **su_id_de sala** con el valor que obtuvo en la parte anterior.

Solicitudes de importación.

```
access_token = 'your_token_here'
room_id = 'your_room_id'
url = 'https://webexapis.com/v1/memberships'
headers = (
    'Authorization': 'Bearer {}'.format(access_token),
    'Content-Type': 'application/json'
)
params = {'RoomId': room_id}
res = requests.get(url, headers=headers, params=params)
print (res.json ())
```

- Guarde y ejecute el archivo. Debería obtener una respuesta similar a la siguiente. Usted debe ser el único miembro a menos que haya agregado a alguien en la aplicación Webex. Los valores de ID se han truncado.

```
devasc @labvm: ~/labs/devnet-src/webex-teams$ python3 list-memberships.py
{'items': [{'id': 'Y2l... rJzg', 'RoomId': 'Y2l... gnm', 'personId': 'Y2l... M2U',
'PersonEmail': 'user@example.com', 'PersonDisplayName': 'PersonDisplayName',
'PersonOrgID': 'Y2l... MGY', 'IsModerator': False, 'isMonitor': False, 'creado': '2020-06-04T 16:50:19 .819Z'}]}
devasc @labvm: ~/labs/devnet-src/webex-teams$
```

Paso 3: Localizar e investigar la documentación de API para su publicación en la API de Membresía.

- Vuelva al sitio web developer.webex.com. En **Referencia de API**, haga clic en **Membresías**, si es necesario.
- La API de Membresías tiene un método POST para **Crear una Membresía**. Haga clic en el vínculo de **Parámetros de consulta** para consultar cuáles están disponibles. En el script, usará los parámetros requeridos **RoomID** y **PersonEmail**.

Paso 4: Utilizar una secuencia de comandos de Python para crear una membresía, agregando a alguien a la sala.

Para este paso, necesitará el correo electrónico de otra persona que sea un usuario registrado de Webex en su organización. Puedes usar el mismo correo electrónico que utilizaste anteriormente para enumerar los detalles de una persona.

- En Código VS, haga clic en el archivo **create-membership.py**.
- Coloque el siguiente código en el archivo. Reemplace **su_token_aquí** con su token de acceso personal. Reemplace **su_id_de sala** con el valor que obtuvo en la parte anterior. Reemplace **new-user@example.com** por el correo electrónico de la persona que desea agregar a la sala.

Solicitudes de importación.

```
access_token = 'your_token_here'
room_id = 'your_room_id'
person_email = 'new-user@example.com'
url = 'https://webexapis.com/v1/memberships'
headers = (
    'Authorization': 'Bearer {}'.format(access_token),
    'Content-Type': 'application/json'
)
params = {'RoomId': room_id, 'personEmail': person_email}
res = requests.post(url, headers=headers, json=params)
print (res.json())
```

- Guarde y ejecute el archivo. Debería obtener una respuesta similar a la siguiente. Usted debe ser el único miembro a menos que haya agregado a alguien en la aplicación Webex. Los valores de ID se han truncado.

```
devasc @labvm: ~/labs/devnet-src/webex-teams$ python3 list-memberships.py
{'items': [{'id': 'Y2l... rJzg', 'RoomId': 'Y2l... gnm', 'personId': 'Y2l... M2U',
'PersonEmail': 'user@example.com', 'PersonDisplayName': 'PersonDisplayName',
'PersonOrgID': 'Y2l... MGY', 'IsModerator': False, 'isMonitor': False, 'creado': '2020-06-04T 16:50:19 .819Z'}]}
devasc @labvm: ~/labs/devnet-src/webex-teams$
```

Parte 7: Administrar mensajes en equipos de Webex.

En Webex, un mensaje puede contener texto sin formato, Markdown o un archivo adjunto. Cada mensaje se muestra en su propia línea junto con una marca de hora y la información del remitente. Puede utilizar esta API de Mensajes para enumerar, crear y eliminar mensajes. En esta parte, enviará un mensaje a la sala que creó en este laboratorio.

Paso 1: Buscar e investigar la documentación de la API para los mensajes.

- Vuelva al sitio web developer.webex.com. En **Referencia de API**, haga clic en **Mensajes**.
- Explore las distintas llamadas API que puede realizar con la API de **mensajes**.
- Haga clic en la solicitud POST para **Crear un mensaje** y explore los **parámetros de consulta**. Puede dar un aviso para un mensaje de texto simple, puede utilizar el parámetro **text** o **markdown**. En este paso, especificará un mensaje con formato Markdown. Busque en Internet para obtener más información sobre Markdown.

Paso 2: Utilizar una secuencia de comandos de Python para enviar un mensaje a una sala Webex.

- a. En Código VS, haga clic en el archivo **creat-markdown-message.py**.
- b. Coloque el código siguiente en el archivo. Reemplace **su_token_aquí** con su token de acceso personal. Reemplace **su_id_de sala** con el valor que obtuvo en la parte anterior.

Solicitudes de importación.

```
access_token = 'your_token_here'
room_id = 'your_room_id'
message = 'Hello **DevNet Associates**!!'
url = 'https://webexapis.com/v1/messages'
headers = (
    'Authorization': 'Bearer {}'.format(access_token),
    'Content-Type': 'application/json'
)
params = {'RoomId': room_id, 'markdown': message}
res = requests.post(url, headers=headers, json=params)
print (res.json ())
```

- c. Guarde y ejecute el archivo. Debería obtener una respuesta similar a la siguiente. Observe que la rebaja se convirtió a HTML. Los valores de ID se han truncado.

```
devasc @labvm: ~/labs/devnet-src/webex-teams$ python3 create-markdown-
message.py
{'id': 'Y2l...RjZg', 'roomId': 'Y2l...GNm', 'roomType': 'group', 'text': 'Hello DevNet
Associates!!', 'personId': 'Y2l...M2U', 'personEmail': 'user@example.com', 'markdown':
'Hello **DevNet Associates**!!', 'html': '<p>Hello DevNet
Associates!!<strong></strong></p>', 'created': '2020-06-04T19:08:49.145Z'}
```